

OpenRAM-Based 4KB SRAM Investigation

Author Name: Vijay kumar

Abstract—This Week-1 investigation focused on understanding the generation flow of a 4KB SRAM (1024×32) using OpenRAM and SKY130 technology. The study explored SRAM organization, 6T bitcell operation, OpenRAM compiler hierarchy, and technology integration. The primary observation was that OpenRAM assembles reusable custom cells through a hierarchical compiler flow rather than generating a flat transistor-level memory design.

Keywords—SRAM, OpenRAM, SKY130, Memory Com-piler, VLSI

I. INTRODUCTION

The objective of this investigation was to understand how OpenRAM converts a user-defined memory specification into a technology-aware SRAM implementation. Rather than targeting final SRAM generation, the study focused on compiler organization, technology integration, expected outputs, and common setup challenges.

II. SRAM ARCHITECTURE INVESTIGATION

The reference design is a 1024×32 SRAM with a total capacity of 32768 bits (4KB). Investigation of the architecture showed that OpenRAM organizes memory using a bitcell array, address decoding logic, wordline drivers, write circuitry, sense amplifiers, and output buffers.

The 6T SRAM bitcell was studied as the fundamental storage element. During read operation, precharged bitlines develop a small differential voltage that is detected by the sense amplifier. During write operation, complementary values are forced onto the bitlines through the write driver while the selected wordline connects the cell to the bitlines.

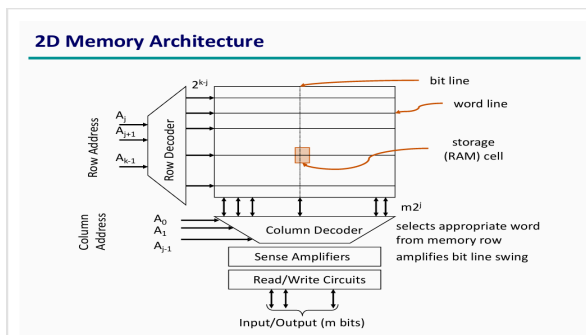


Fig. 1. SRAM architecture showing address decoder, SRAM cell array, precharge circuitry, write circuitry, sense amplifier, and output buffers.

III. OpenRAM COMPILER INVESTIGATION

Source-code exploration showed that OpenRAM follows a hierarchical design methodology rather than a monolithic SRAM generation flow. The observed hierarchy can be summarized as:

Configuration $\rightarrow$ OPTS $\rightarrow$ sram_config $\rightarrow$ sram.py
 $\rightarrow$ sram_1bank $\rightarrow$ bank
 $\rightarrow$ bitcell array + peripheral circuits

The configuration file defines memory parameters such as word size, memory depth, technology name, and process corners. OpenRAM uses these inputs to determine the internal memory organization and assemble the SRAM using reusable modules. A key observation was that modules such as bitcell_1port.py are compiler-side wrappers around technology-provided custom cells. The actual bitcell, sense amplifier, write driver, and supporting layouts are supplied by the SKY130 technology library.

IV. SKY130 INTEGRATION AND DEBUG ANALYSIS

The SKY130 technology package acts as the interface between the generic OpenRAM compiler and the fabrication technology. Important components identified during the investigation include: gds_lib – Custom-cell layouts, sp_lib – SPICE netlists, models – Transistor model files, tech.py – Technology abstraction and design rules. The SRAM generation flow starts from a configuration file and produces outputs such as SPICE, GDS, LEF, Liberty, and Verilog models. Potential setup issues identified during the investigation include incorrect OpenRAM environment variables, technology-path mismatches, missing custom-cell views, missing model files, and SPICE pin-order inconsistencies. A practical debugging strategy is to verify technology integration and custom-cell availability before attempting larger SRAM generations.

V. FLOW ANALYSIS

The SRAM generation flow begins with a Python configuration file such as myconfig_sky130.py. The main inputs include word size, number of words, technology name, number of ports, supply voltage, process/model corner settings, and output location. These inputs do not directly create a physical array; they first pass through OpenRAM's sizing and hierarchy-building stages.

The expected outputs from the flow include SPICE netlists, GDS layout, LEF abstract, Liberty timing model, Verilog model, LVS SPICE, configuration copy, and optional extracted/PEX-related files. Even though full macro generation was not required in Week-1, identifying these outputs helped clarify the purpose of each stage in the compiler flow.

VI. KEY OBSERVATIONS

The most significant observation was that OpenRAM is fundamentally a custom-cell-based compiler. Rather than generating every circuit element from scratch, it assembles verified technology cells into larger SRAM structures. A second observation was that SKY130 acts as a technology abstraction layer. By separating technology-specific information from compiler logic, OpenRAM can reuse the same architecture while supporting different fabrication technologies.

VII. REFERENCES:

1. OpenRAM Project, "OpenRAM Documentation and Source Code."
2. Google and SkyWater, "SKY130 Open Source PDK Documentation."
3. https://github.com/ShonTaware/SRAM_SKY130